

Requirements and prototyping

IT Product Development

Innopolis University, 2026

Agenda

- Why this week
- The chain: vision, stories, criteria, prototype
- Vocabulary
- Validating with the customer

Why this week

Last week

You settled:

- Which products exist, and how well each one does the job
- Where they all fall short: GAP-01, GAP-02
- What you will do differently: VP-01, VP-02
- What the customer disagrees with

You did not settle:

- What your product actually has to do
- What it will not do
- What you are building first

This week: say what it must do

The research said **what to build**.

This week you say **what it must do**, and you find out how much of that was wrong.

Conjectures lead to experiments. Experiments lead to facts. Facts lead to decisions.

Quiz

- What did last week's research give you that you can build on?
- Name one thing it did not give you.

The chain

The chain

GAP-nn, VP-nn -> the vision -> **US-nn** -> the prototype -> the customer -> **a change**

Every link answers one question:

- The vision: what is this for?
- A user story: what does one user need?
- Acceptance criteria: how would we know?
- The prototype: are we guessing?
- The meeting: where are we wrong?

The product vision

The vision is the one page a reader holds to understand what you decided to build. It has five parts:

- The **goal**: what the product must achieve
- The **constraints**: what you cannot change, and what each one costs
- The **stakeholders**: who uses it, who operates it, who is affected
- The **boundary**: what it will not do
- A **system context diagram**: your system in the environment

The goal traces to a VP-nn. A goal that supports no value proposition is a feature you would like.

Constraints and assumptions

Constraint: a condition you cannot change

Assumption: a belief you have not verified

Record every constraint under one of three sources:

- **Customer-given:** imposed by the customer or the project catalog
- **Team-given:** e.g. the team size, the weeks left in the course, skills
- **Derived:** what follows from the other two

For each constraint, say what it costs you. A constraint that costs nothing is not yet understood.

The boundary

Boundary: what your product will **not** do.

It is what makes the context diagram checkable.

With it, a reviewer can point at an actor and ask which of your exclusions rules them out.

Without it, a diagram can only be judged as looking reasonable, and “reasonable” is not a test.

The context diagram

Context diagram: is a picture of the product at its edges:

- The **product** itself
- The **external actors** that use it or are affected by it
- The **external systems** it exchanges data with

It stops there. No containers, no components, and no internal structure; that is for later weeks.

Every actor on the diagram must survive your boundary. If an actor has no reason to exchange anything with the product, it is not on the diagram.

A user story

User story: captures a need of an actor together with the value.

As a <user>
I want to <action>
so that <value>

A user is any actor with a goal: the person the product serves, and the operator or administrator who keeps it running.

It is a **need**, not a design. “See the sessions I missed this week” is a need you can still be wrong about. “Add a calendar page” is a feature you have already built in your head.

Naming design specifics may constrain developers who will work on the user story.

If you cannot finish a user story with a value, you have written a task.

One story, twice

Both are about a study group planner. Both are made up.

Not a story:

| As a student, I want a calendar page so that I can see all my study sessions in one place.

This is a feature description (a screen)

A story:

| As a student preparing for an exam, I want to see the sessions I have already missed, so that I can decide which one to reschedule before the week is over.

This one has a person in a situation, an object they need, and a decision they are trying to make.

Two traps worth naming

Trap 1: a solution wearing a story's clothes. The story names a screen, a button, or a component. You have already designed it, so you are constrained by the design.

Trap 2: a goal nobody can fail. “Be fast”, “be reliable”, “delight the user”. Nobody can tell on Friday whether you achieved it, so it cannot guide a decision on Wednesday.

Acceptance criteria

The story says **why**. The criteria say **what success looks like**.

Two or more per story, and the test is simple: **could somebody who is not you run the check and get the same answer?**

A criterion may name a screen, a field, or a system state, because that is what an observer sees. The story may not, because that would be the design.

“Works well” is not a criterion. A criterion you cannot run is a belief.

A criterion that is hard to write is not a writing problem. It means you do not yet know what the product does in that case, which makes it a good thing to prototype.

Prioritizing with MoSCoW

Priority is a four-way scale, relative to the product you intend to finish:

- **Must Have:** the product is not the product without it
- **Should Have:** important, but the product is still coherent without it
- **Could Have:** valuable, and the first thing to cut if can't implement for some reason
- **Won't Have:** a real need you have deliberately excluded

Each label should be chosen deliberately. Whoever disagrees that a feature/user story is **Must Have** has to say why the product can still be the product without it.

A **Won't Have** is written down with its reason, not quietly left out. A reader can argue with a recorded exclusion; they can only guess about a missing one.

The prototype

A prototype is an instrument for finding out what is wrong and for killing wrong ideas early.

It is not a showcase, and it is not the product.

Ask it one question first: **which story am I least sure about?** Then build the cheapest thing that gets a reaction to that question.

Forms of prototype

Match the fidelity to the question, not to your ambition.

- **Paper or static image:** fastest, for layout, vocabulary, and “is this the right problem”
- **Clickable design:** for a flow you want the customer to move through
- **Code spike:** for a technical risk, kept off main and thrown away after

A high-fidelity prototype is the wrong tool for “does the customer recognise this problem”: you will spend two days on it and they will comment on the colour.

Quiz

- What question does each artifact in the chain answer?
GAP-nn, **VP-nn** -> the vision -> **US-nn** -> the prototype
- What is the difference between a need and a design?
- How do you tell a real acceptance criterion from a wish?

Vocabulary

Main terms

Goal: what the product must achieve

Boundary: what the team will not do

Constraint: a condition you cannot change

Assumption: a belief you have not verified

Story: one thing a user needs

Criterion: how you would know you delivered it

Prototype, proof-of-concept, MUP, product

Proof-of-concept: “does this work at all technically?”

Prototype: “does this deliver the value?” Normally gets thrown away

Minimum usable product: real code that delivers value to somebody

The product: what you ship, weeks later

Quiz

- Which of the six words can be wrong, and how would you find out?
- What is the difference between a prototype and a MUP?

Validating with the customer

What the second meeting asks

The target is one sentence: **which of these is wrong?**

The prototype, the boundary, and the candidate for the minimum usable product.

The rest of the stories only if time allows.

Not the business goals. Those were settled last week. A later meeting settles one thing or two, and everything else is a conversation you do not need to have again.

Do not treat agreement as a result

“That sounds great” about your own idea has told you almost nothing.

If the customer agrees with everything, you showed them the answer rather than the question. The Disagreements table in your meeting report should not be empty.

Warning! A prototype that validated everything proved nothing, because you chose the parts you were already sure about.

Quiz

- What is the target of this meeting, in one sentence?
- What do you do when the customer approves everything?

Review questions

- What does the chain give you that a list of features does not?
- What is the boundary for, and what does it make checkable?
- How do you tell a real acceptance criterion from a wish?
- Which of your stories would you prototype first, and why that one?

Questions?